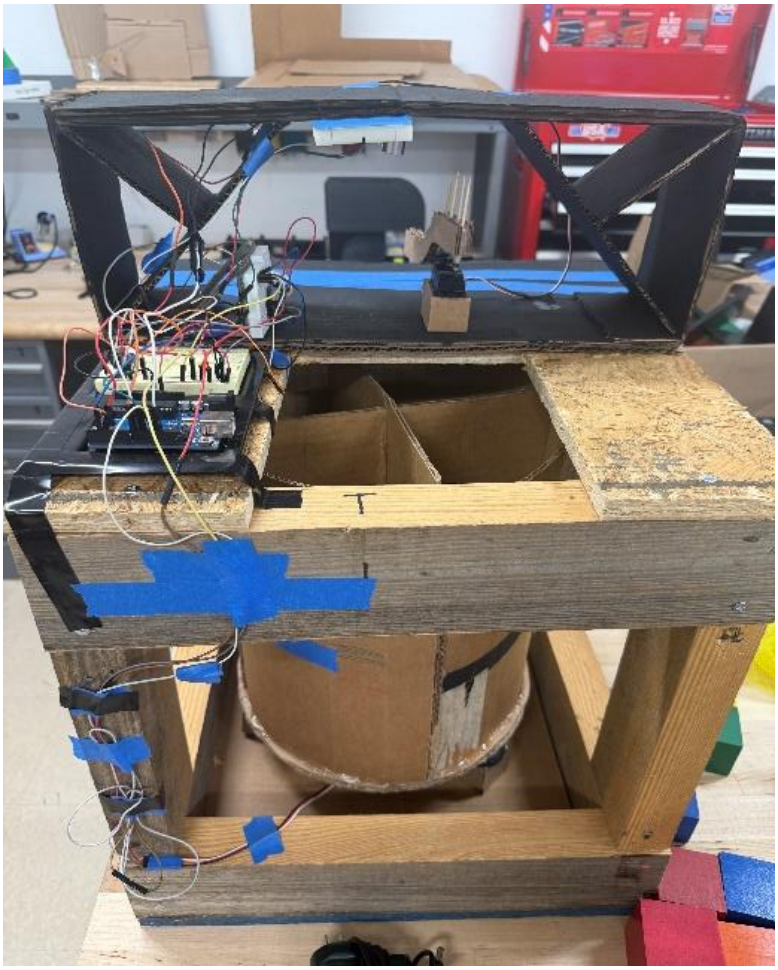
Automated Cube Sorting Machine



Austin Balcom
William Bowling
Ian Day
Mia Iampieri
Timothy Martinez
Valerie Simonds

Mathematics and Engineering- EGR-1100
College of Southern Maryland
December 16th, 2025

Automated Cube Sorting Machine

Austin Balcom
William Bowling
Ian Day
Mia Iampieri
Timothy Martinez
Valerie Simonds

Mathematics and Engineering- EGR-1100
College of Southern Maryland
December 16th, 2025

Summary

On September 18, 2025, Company X presented Team 2 with the need to automate the quality assurance of 2-inch wood blocks they produce. Company X has named the following requirements for the sorting process identifies and bins cubes that do not meet the size specification, identifies and bins cubes that meet the size specification but not the color specification, sorts and bins cubes that meet both specifications by color. Also providing the following cube specifications, a cube measurement of 2.00” ± 0.10” and that the cube color is green, yellow, or orange.

To accomplish this task, Team 2 has presented an automated cube sorting machine based on a rotating base design. Made of three main components: the frame, sensor subassembly, and sorting bins mounted on a rotating base. The frame of the machine will be a 16”x16” cube built of 2x4s and plywood. Mounted on the top of the frame is the sensor subassembly, where the cube is first placed. Once the color and size have been detected, the bins attached to the base spin to position the correct bin under the block. When the correct bin has been positioned under the block, a motor with an arm attachment will push the block into the correct bin.

In initial testing, the cube sorting machine was able to sort 9 out of 10 cubes in the 4-minute time limit. To improve speed, the team modified the code, including shortening delay times and sensing color first to prevent measuring the size when the cube is the incorrect color. In final testing, this modification increased the speed of the machine, sorting the 10 blocks in 3 minutes. During the presentation, the machine was effective by accurately sorting 10/10 wooden blocks by size and color within the 4-minute time limit.

Contents

Introduction

On September 18, 2025, Company X presented Team 2 with the need to automate the quality assurance of 2-inch wood blocks they produce. Currently being sorted manually, the 2-inch blocks must be sorted by color and size. With no human intervention, management of company X has identified the following requirements for the separation process:

1. Identifies and bins cubes that do not meet the size specification
2. Identifies and bins cubes that meet the size specification but not the color specification
3. Sorts and bins cubes that meet both specifications by color

Also providing the following cube specifications:

1. Cube measurement 2.00" ± 0.10"
2. Cube color is green, yellow, or orange

On December 11, 2025, Team 2 will present a completed cube sorting machine for a demonstration with the following system requirements:

1. System shall use an Arduino Uno as the microcontroller.
2. Power within the system "boundary" shall be no greater than 12VDC. 120VAC power, if used, shall be converted to no greater than 12VDC prior to crossing the system "boundary".
3. System shall correctly bin ten random cubes within four minutes. The set of cubes includes:
    a. Cubes that meet the size specification and the color specification
    b. Cubes that do not meet the size specification that are as large as 2.30" or as small as 1.70"
    c. Cubes that meet the size specification but do not meet the color specification.
4. The bins are part of the system and therefore part of the design. Separate bins are required for the cubes that meet the size and color specifications – i.e., one bin for green, one bin for yellow, one bin for orange. One or more bins can be used for cubes that do not meet the size and/or color specifications. Each bin shall be capable of holding a minimum of four cubes.
5. The system, including the bins, shall occupy a volume no larger than 8 cubic feet.

Considering this Team 2 conceptualized the following three designs:

1. Chute sorting machine: Gravity fed chute; cubes move past color and size sensor to a series of gates that would drop each block to their respective bin. Positive aspects of this design are the simplicity of less moving parts and not having to impart energy to move the blocks. Challenges for this design are alignment of the cubes for proper sensing and possibility of cube jams.
2. Rotating base/bins: A machine with the sensor assembly mounted above a rotating base. Positive aspects of this design are the visually engaging nature of the machine and low probability of jams or blockage. Negative aspects of this design are the multiple moving parts and precise calibration of the base.

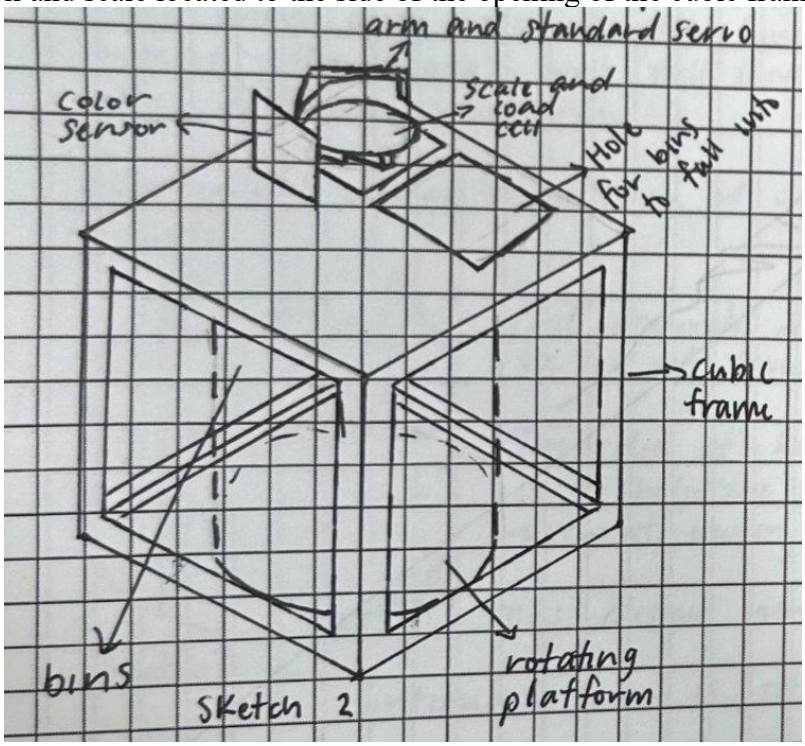
3. Conveyor belt system: Belt moves cubes past the senor assembly to their respective bin where an arm pushes the block to the bin. Positive aspects of this design are faster sorting due to continuous operation. While negative aspects of this design are slippage issues and positioning of sensors.

Analyzing these three designs, Team 2 selected the rotating base as the final design. Initially thought of as complex with multiple moving parts, it was discovered that the cube itself would move only once. With correct positioning of the base, the possibility of jams or blockage is seemingly low.

Discussion

The final design consists of three main components: the frame, sensor subassembly, and the sorting bins mounted on a rotating base. First, the cube is placed into the sensor subassembly where the color and size are detected. Once the color and size have been detected, the bins attached to the base spin to position the correct bin under the block. When the correct bin has been positioned under the block, a motor with an arm attachment will push the block into the correct bin.

Figure 1: Earlier design prior to using the ultrasonic sensor. The sketch includes the load cell and scale located to the side of the opening of the cubic frame.



Engineering Theory

The frame of the machine measures 16”x16” and is built of 2x4s and plywood. The frame was built to this size to have the space to contain the necessary components for the machine to properly function, while maintaining space to make changes and perform repairs. The frame

materials were chosen to provide a higher level of protection than materials like cardboard or plastic. While also being cost efficient compared to steel or other materials.

Mounted on the top of the frame is the sensor subassembly, where the cube is first placed. The sensor subassembly consists of two sensors and a motor. The first sensor is a DFROBOT TCS3200 RGB Color Sensor for Arduino. This sensor uses an 8 x 8 array of photodiodes with different filters to convert light into current. The current produced is then converted into a square wave whose frequency is proportional to the light intensity of the chosen color. This finds the color of the block. The second sensor is the HC-SR04 Ultrasonic sensor. It works by sending sound waves from the transmitter, which then bounce off an object and return to the receiver, finding the block's size. Lastly, the sensor subassembly contains a motor with an arm attachment, used to knock the block into the correct bin once the size and color have been determined.

To keep the program manageable, each task was broken into its own function. Instead of letting all the logic pile up inside the main loop, the code was organized into small, clearly named functions such as checking for cubes, evaluating the cubes that were found, determining their color, verifying their size, and spinning the servos to sort them. Separating these processes made the program significantly easier to read and debug, especially while dealing with unreliable sensor data. The function responsible for checking for cubes only focused on validating distance readings, and the function that checked color handled sensor inconsistencies and lighting issues. The servo movement was placed in its own function so that movement logic was never mixed with sensor logic. This structure improved readability and made it more convenient to adjust or replace individual parts of the system without rewriting large sections of code.

The last component is the rotating base, consisting of a 270° motor attached to an 11" circular cardboard base, supported by 6 caster wheels. The caster wheels were added to provide a more stable base and reduce the risk of failure between the motor and platform. Mounted on the cardboard base are 4 sorting bins. This includes a discard bin and 3 bins for the wooden blocks colored green, yellow, or orange that are the correct size. The discard bin is positioned in the resting position located directly under the opening connecting the senser subassembly. Therefore, if a cube is measured to an incorrect size, it can immediately be discarded and reduce overall performance time.

Problems Encountered

The first problem encountered was with the sensor that measured the size of the block. In the original design, a scale was used to determine the size of the block. An unresolvable issue came from the scale in not returning a value for the block. The team hypothesized that the problem was in the wiring circuit of the load cell but could not properly identify the problem. To overcome this, Team 2 replaced the scale with an alternative sensor to determine size; using the ultra-sonic sensor.

The second problem faced during the build of the cube sorting machine was in the color sensor returning a color value for red and orange that did not match the original value determined. This problem was determined to arise from changing ambient lighting when the machine was reading color values. To solve this, Team 2 built a housing cover to give the color sensor more consistent levels of lighting.

The third problem presented itself in the revolving base, when 5 to 6 cubes had been sorted the weight made the base unable to revolve. The solution found came with adding more caster wheels to the base from 3 to a total of 6.

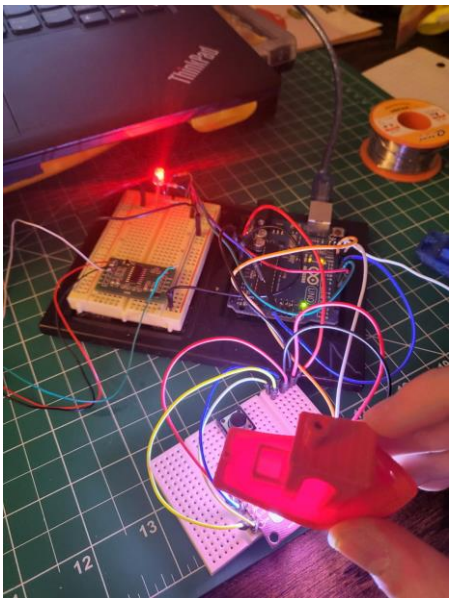
The fourth problem was issues with the code of the automated cube sorter. When programming, getting reliable data from the Ultrasonic Sensor and the Color Sensor was a challenge. Therefore, the code had to have many safeguards put in place to prevent inaccurate data from merging with correct data and affecting the logic-based decisions. Getting the comparison values for the Color Sensor was mostly trial and error, but there was consideration of the lighting of the classroom and it's "red bias." The Ultrasonic Sensor was even more unreliable, so data from the sensor went through several processes. First, the data was averaged between ten readings, then two averaged distances were compared. If they were within 2 mm, then the data could be used for decision-making.

Testing

The testing for this project is broken into three main parts; part testing, assembly testing, and time trial. Testing this way enabled the team to stay focused on the project's objectives, while completing the machine by the required deadline. Part testing included the color and sizing sensors, rotating arm motor, and base motor. The purpose of assembly testing was to have the machine complete a full cycle, sorting a wooden block incorporating all components. Lastly, a time trial was conducted to confirm the ability of the machine to sort 10 wooden blocks within the required time limit.

During part testing, the results for the rotating arm motor and base motor were successful only requiring minor adjustments. When testing the load cell (sizing sensor), there was no data returning to the Arduino requiring replacing it with the ultra-sonic sensor. When testing the color sensor, intermittent false color readings required adding a housing cover and painting it black to reduce ambient lighting. This also doubled as a place to position the ultra-sonic sensor.

Figure 2: Color Sensor Testing



Assembly testing consisted of incorporating all the components to run a full sorting cycle. Successfully completing the earlier testing phase meant that mostly small adjustments were needed for the code for the machine to successfully run a sorting cycle. The largest problem met during this phase of testing was the rotating base having problems turning, which was solved by adding additional caster wheels to the base.

During the time trial, the cube sorting machine sorted 9 of the 10 wooden blocks properly within the 4-minute time limit. To improve the speed, team 2 changed the code for the machine to have less delay between functions. Additionally, to save time, change the operation order for sensing color and measuring size. If the wooden block is the incorrect color, it will bypass checking the size of the cube. After this adjustment, the 10 wooden blocks were correctly sorted in 3 minutes and 1 second.

Conclusion

Assigned the problem of automating the quality assurance of sorting wooden blocks for Company X. Team 2 presented an automated wooden block sorting machine that is a rotating base design. This design was chosen based on the limited movement of the wooden block, with the wooden block moving once after being initially placed. Testing of the cube sorting machine uncovered problems with the load cell, color sensor, and stability of the rotating base. This led to team 2 changing the load cell to an ultra-sonic sensor to measure the wooden block size. Add housing to the sensor array to block out unwarranted ambient lighting. Lastly, increasing the number of caster wheels supporting the vase from 3 to 6. The results from the final presentation demonstrated the cube sorting machines' effectiveness with all of the objectives being met within the guideline requirements. Even while meeting all the requirements, some improvement to the sorting machine would be higher quality components, an improved power system, and adding the load cell. Higher quality components would be 3-D printing the bins and sensor subassembly housing. Improvements to the power system would fix power issues; that at present does not affect performance. Contributing factors toward the success of Team 2 would be time management, work delegation, and adaptability. The setting of team deadlines and strictly adhering to them kept the project on schedule. Properly delegating work throughout the team

kept everyone engaged and prevented team members from being overwhelmed. Adaptability of the team to redesign the components that did not function properly while being able to adapt to problems as they arose, gave the team the flexibility to present an automated cube sorter able to meet the expected requirements.

Appendix A: 2D Engineering Drawings of Created Parts

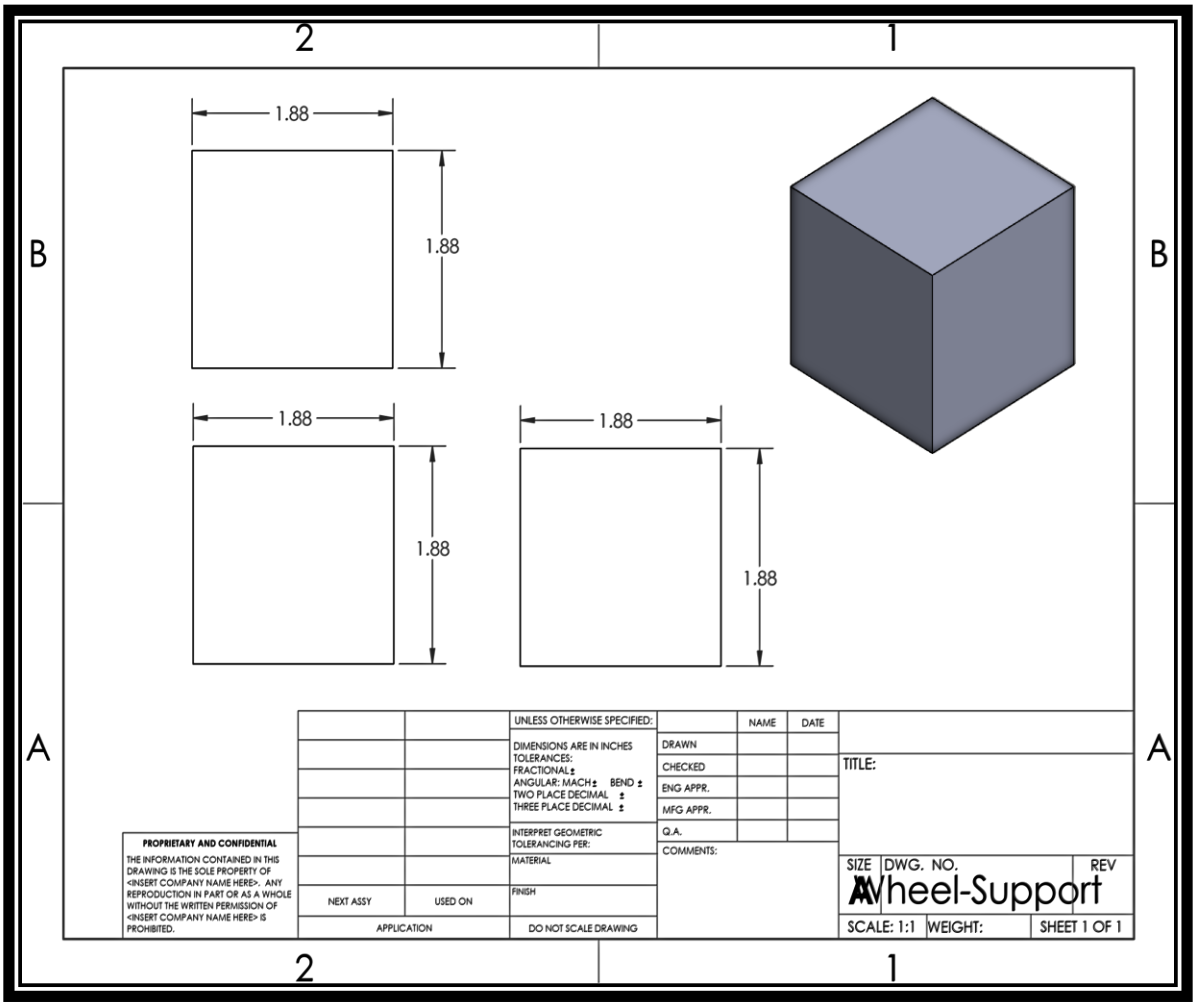


Figure A-1: Above is the base that the caster wheels sit on that provide support to the base of the rotating base subassembly

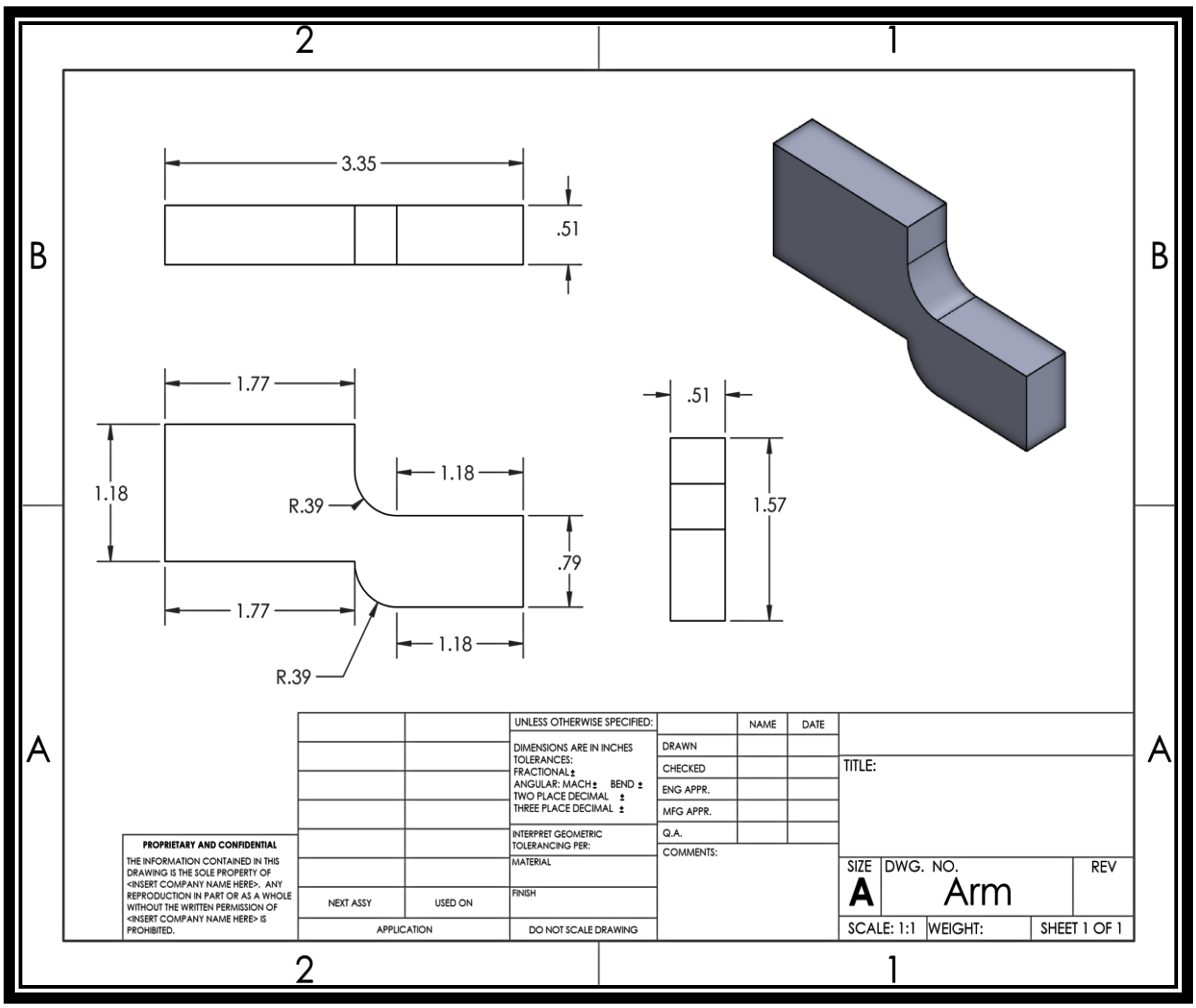


Figure A-2: Above is the arm that is mounted on the motor located in the sensor subassembly that pushes the wooden blocks into the sorting bins.

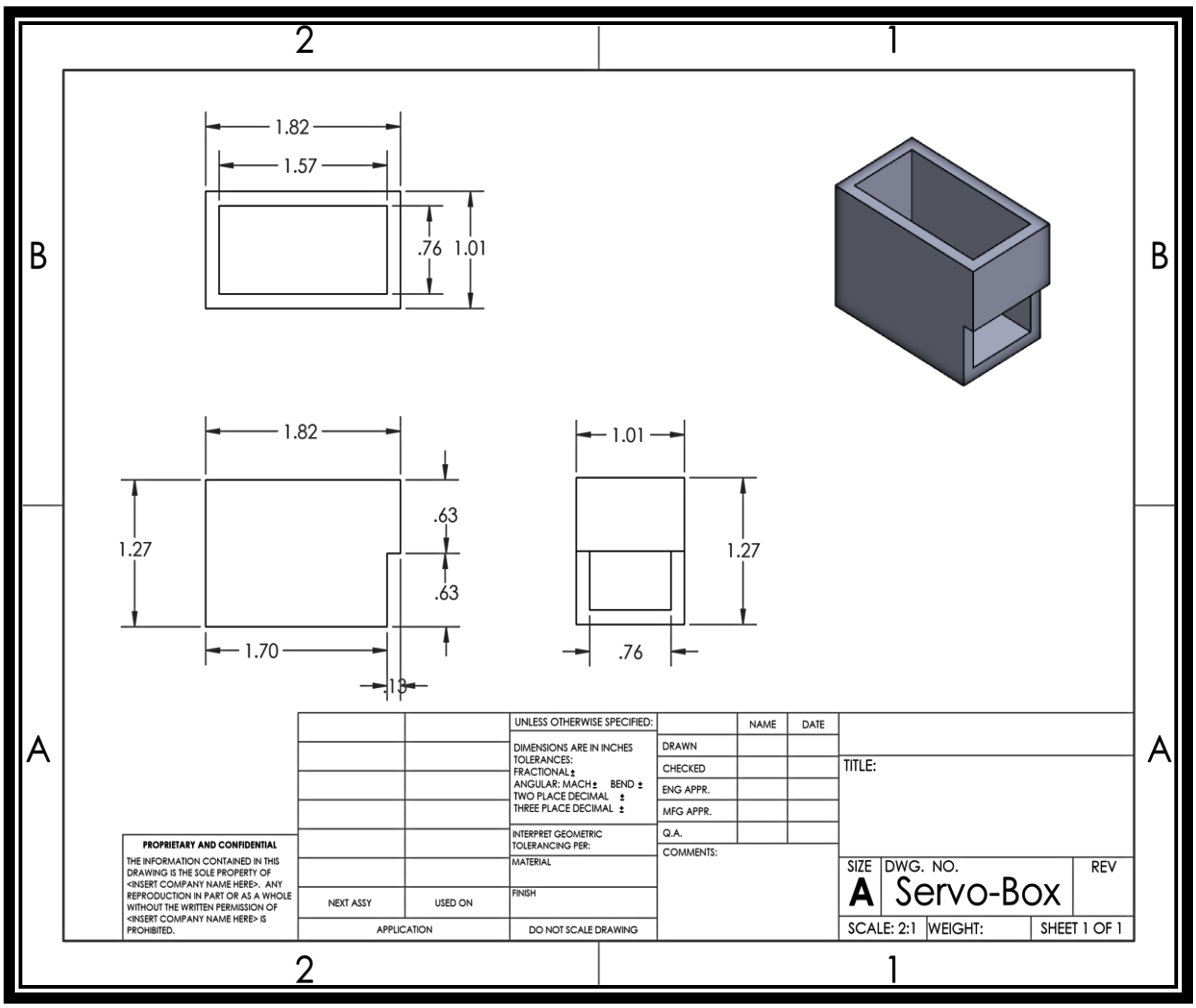


Figure A-3: Above is the housing that both servo motors (sorting arm, rotating base) sit in to provide stability and allow attachment to the machine.

Appendix B: System Assembly Drawings

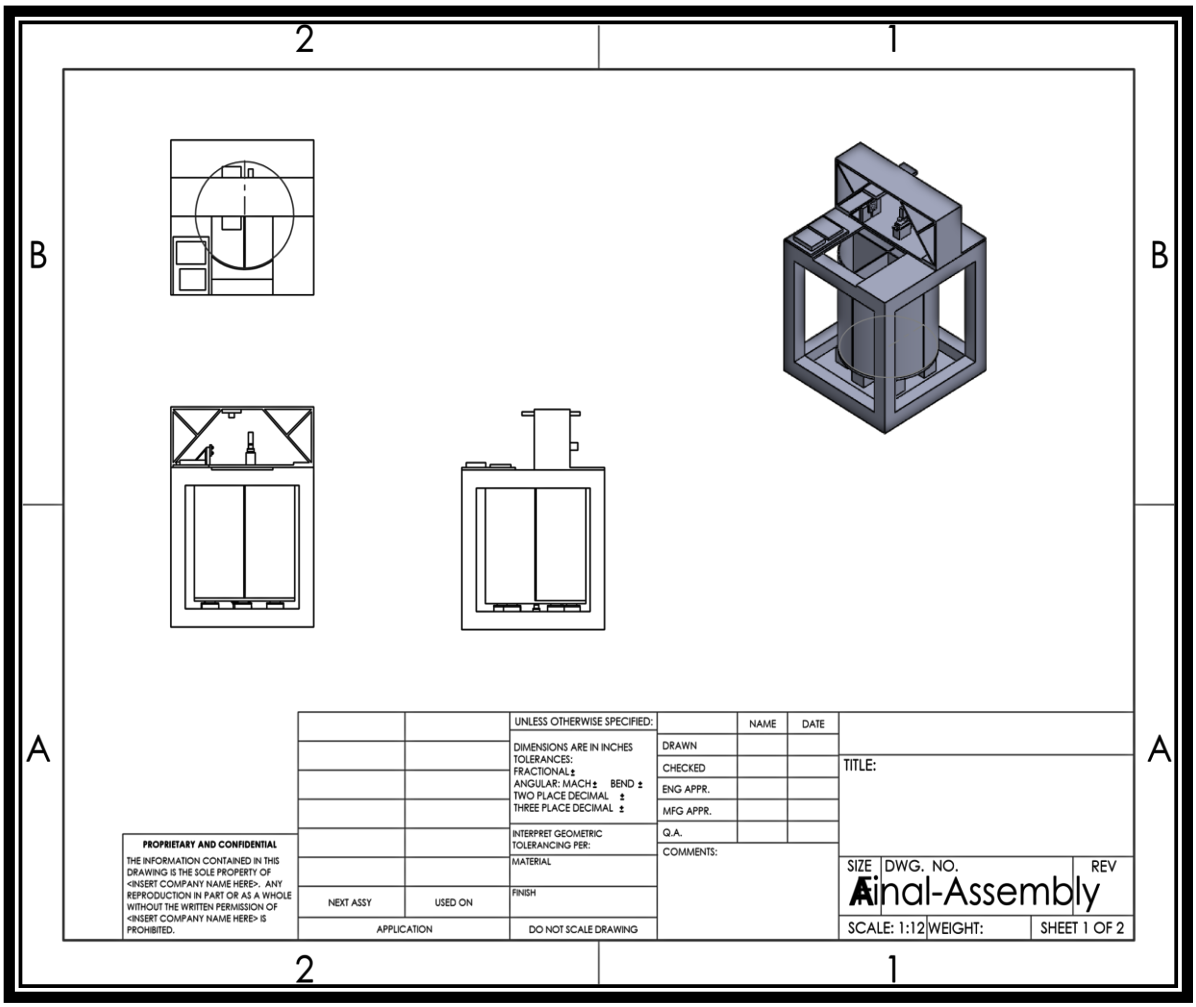


Figure B-1: Complete assembly drawing of the cube sorting machine including the Top, Front, Side, and Isometric view.

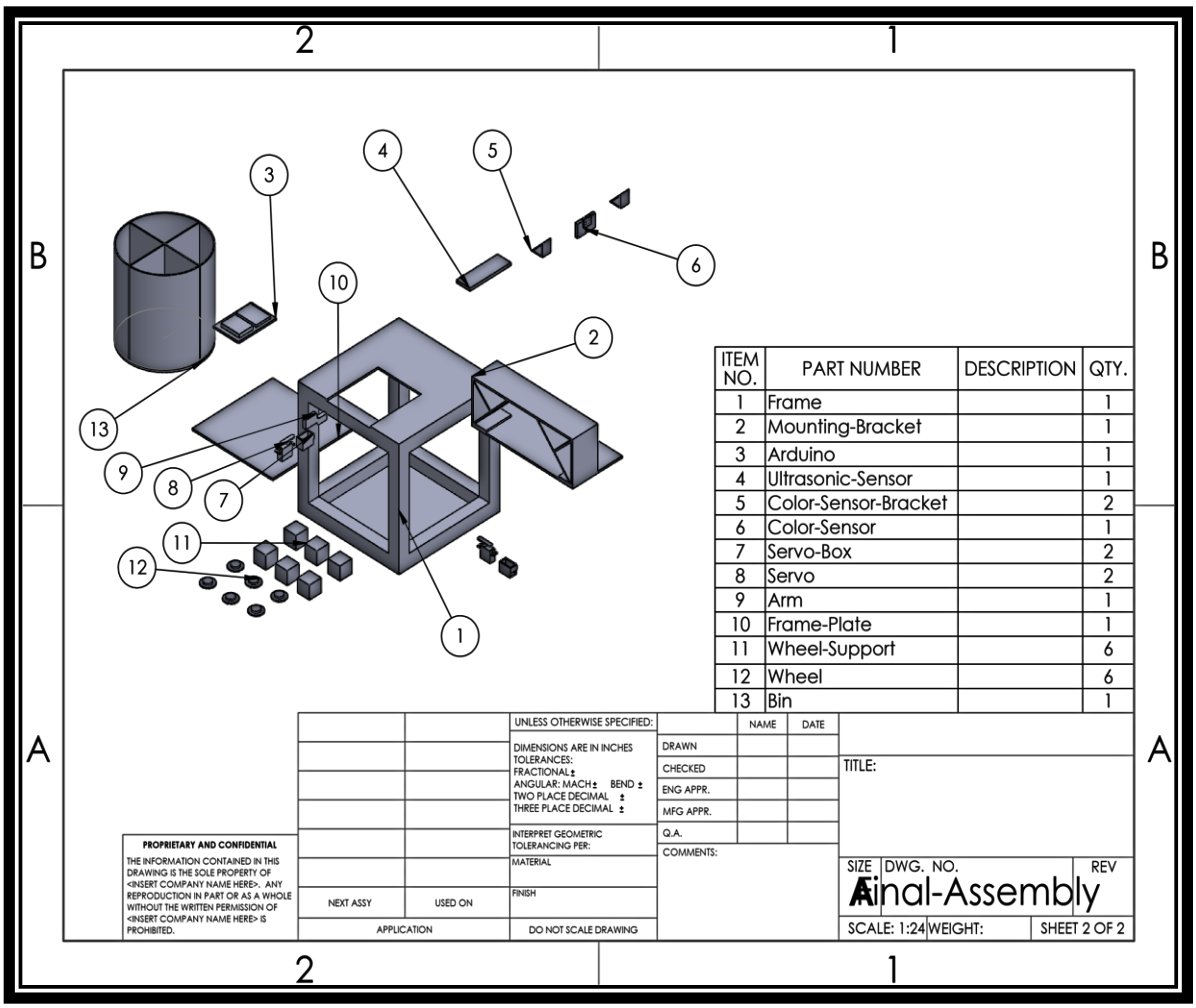


Figure B-2: Drawing including showing the exploded view with the bill of materials.

Appendix C: Electrical Schematic

Schematic details of the connections of all components of the machine. Everything is connected and controlled via the Arduino pins and has power and GND supplied by the Arduino's 5V and GND pins. Use an external battery source due to too many connections on the 5V Pin. Too many connections on a singular 5V pin can reduce Voltage efficiency that powers all components. The software used is Circuit Diagram.

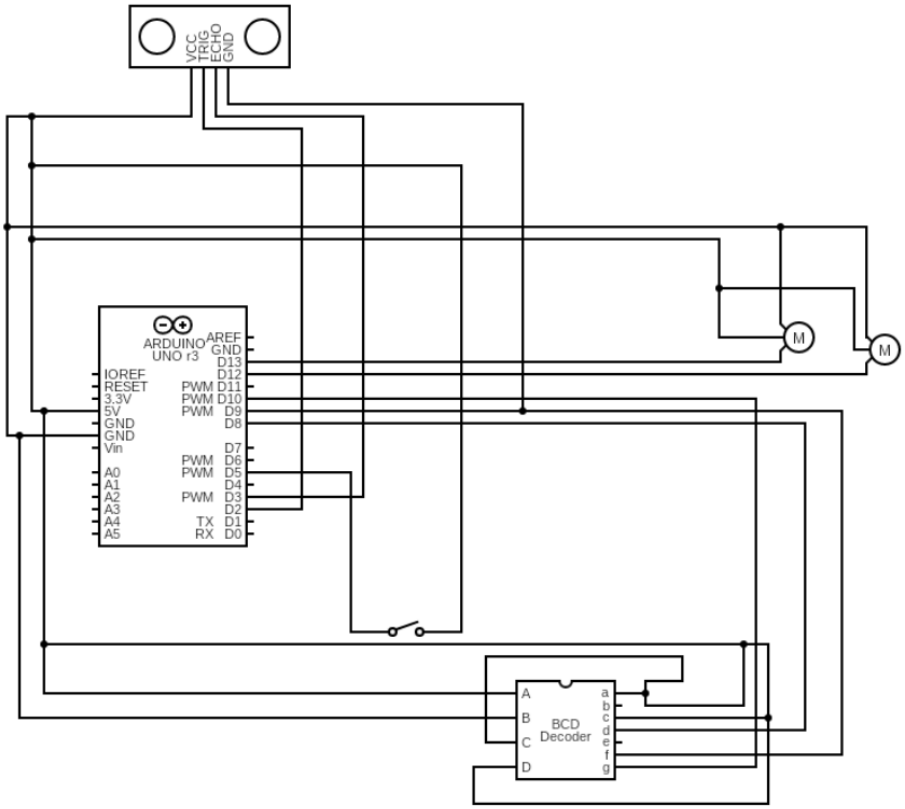


Figure C-1: Electrical schematic of the Arduino Uno R3**,** Ultra Sonic Sensor (USS), and [BCD Coder = Color Sensor] using components such as Dig → (Pin D2), Echo → (Pin D3), Gnd → (Pin 9), VCC → (Pin 5V), Rotating Bins Motor → (Pin 12), Rotating Arm Motor → (Pin 13), Toggle Switch (Button) → (Pin D5).

Appendix D: Microcontroller Source Code

```
#include <Servo.h>

// Color Sensor Pins
const int S2 = 8; //establishes integer variables for Arduino I/O pins
const int S3 = 9;
const int sensorOut = 10;

// RGB LED Pins
const int red_PIN = 6;
const int green_PIN = 7;

// Ultrasonic Sensor Pins
const int trig_PIN = 2;
const int echo_PIN = 3;

// Ultrasonic Distance Variables
float duration_us, distance_cm;

float distance_from_floor = 11;
float minimum_cube_size = 2.8;
float maximum_cube_size = 4.2;

// Color Sensor RAW Values
int redRaw = 0;
int greenRaw = 0;
int blueRaw = 0;

// Button
const int buttonPIN = 5;
int buttonState = 0;

// Servo Setup and Pins
Servo servo_base;
Servo servo_arm;

const int servo_base_PIN = 12;
const int servo_arm_PIN = 13;

int base_position = 0;
int arm_position = 0;
```

```
void setup() {
  // Setup Servo
  servo_arm.attach(servo_arm_PIN);
  servo_base.attach(servo_base_PIN);

  // Setup Neopixel Pins
  pinMode(red_PIN, OUTPUT);
  pinMode(green_PIN, OUTPUT);

  // Setup Color Sensor Pins
  pinMode (S2, OUTPUT);
  pinMode (S3, OUTPUT);
  pinMode (sensorOut, INPUT);

  // Setup Ultrasonic Sensor Pins
  pinMode(trig_PIN, OUTPUT);
  pinMode(echo_PIN, INPUT);

  Serial.begin(9600);

  digitalWrite(green_PIN, HIGH);
  digitalWrite(red_PIN, LOW);
}

void loop()
{
  const int wait_time = 20000;

  // Reset Everything to Zero Degrees (Safeguard)
  servo_base.write(0);
  servo_arm.write(0);

  // Checks to see if a Cube has been placed down (As cubes have a size and a color)
  if (get_ultrasonic_dist() < distance_from_floor-1.5 && get_colorsensor_color() != "None"){
    delay(3000);
    evaluate_cube();
  }
  // Read the state of the button value:
  buttonState = digitalRead(buttonPIN);
  Serial.println(buttonState);

  // Check if the pushbutton is pressed. If it is, the buttonState is LOW (It was faster to change to
LOW than rewire)
  if (buttonState == LOW) {
    rotate_servo(servo_base, 0, 135);
    delay(wait_time);
```

```
  }

  // Wait 3sec before detecting another cube
  delay(3000);
}

void evaluate_cube(){
  String cube_color = get_agreed_color();
  float cube_size = 0;

  // If the Cube is not the correct color (Red or Blue) then skip checking the size
  if (cube_color == "Orange" || cube_color == "Yellow" || cube_color == "Green"){
    cube_size = distance_from_floor - get_agreed_dist();
  }

  int base_target_angle = 0; // Default Angle
  int arm_target_angle = 0; //  Default Angle
  delay(200);

  if (cube_color == "Red" || cube_color == "Blue"){
    // Send to Bad Bin
    base_target_angle = 0;
    Serial.println("Cube sent to Bad Bin. Color: ");
    Serial.print(cube_color);
  }
  else if (cube_size < minimum_cube_size || cube_size > maximum_cube_size){
    // Send to Bad Bin
    base_target_angle = 0;
    Serial.println("Cube sent to Bad Bin. Size: ");
    Serial.print(cube_size);
  }
  else if (cube_color == "Yellow"){
    // Send to Yellow Bin
    base_target_angle = 50;
    Serial.println("Cube sent to Yellow Bin");
  }
  else if (cube_color == "Green"){
    // Send to Green Bin
    base_target_angle = 135;
    Serial.println("Cube sent to Green Bin");
  }
  else if (cube_color == "Orange"){
    // Send to Orange Bin
    base_target_angle = 270;
    Serial.println("Cube sent to Orange Bin");
  }
```

```
  // Spin the Base Servo to prepare for cube movement
  rotate_servo(servo_base, base_position, base_target_angle);
  base_position = base_target_angle; // Update current position

  // Spin the Arm Servo to push cube into bin
  rotate_servo(servo_arm, arm_position, 90);
  arm_position = arm_target_angle; // Update current position

  // Hold at the target angle for 1.0sec
  delay(1000);

  // Return smoothly to 0° (origin)
  rotate_servo(servo_base, base_position, 0);
  rotate_servo(servo_arm, arm_position, 0);

  arm_position = 0; // Reset current position
  base_position = 0; // Reset current position

  // Wait 1.0sec before "Restarting Program"
  delay(1000);
}

// Rotates 'servo' from 'pos1' to 'pos2' smoothy
// This function exists to control the speed and as a safe guard (As numerous writes to the servo
would need to fail to stop the machine from working)
void rotate_servo(Servo servo, int pos1, int pos2){
  if(pos1 < pos2){
    for (int i = pos1; i <= pos2; i++) {
      servo.write(i);
      delay(15);
    }
  } else {
    for (int i = pos1; i >= pos2; i--) {
      servo.write(i);
      delay(15);
    }
  }
}

// Returns the distance from the top of the cube within a 0.02cm accuracy
// This is the second safeguard for the ultrasonic distance
float get_agreed_dist(){
  float dist1;
  float dist2;
  bool agreed = false;
```

```
  while(agreed == false){
    dist1 = get_ultrasonic_dist();
    delay(50);
    dist2 = get_ultrasonic_dist();
    if (((dist1-dist2 <= 0.02) && (dist1-dist2 >= 0) || (dist1-dist2 >= -0.02) && (dist1-dist2 <=
0))){
      agreed = true;
    }
  }
  return dist1;
}

// Returns the Average of a 'distances_count' amount of distances to improve accuracy (Takes
Approx. 0.25sec)
float get_avg_dist(){
  const int distances_count = 10;
  float total_distance = 0;

  for(int i=0; i<distances_count; i++){
    total_distance += get_ultrasonic_dist();
    delay(25);
  }

  //return total_distance/distances_count+1;
  return get_ultrasonic_dist();
}

// Returns the Distance From the Ultrasonic Sensor to the Cube in centimeters as a float
float get_ultrasonic_dist(){
  float dist;

  // Generate 10-microsecond pulse to TRIG pin
  digitalWrite(trig_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(trig_PIN, LOW);

  // Measure duration of pulse from ECHO pin
  duration_us = pulseIn(echo_PIN, HIGH);

  // Calculate the Distance
  dist = 0.017 * duration_us;

  // Return float Distance (in centimeters)
  Serial.println(dist);
  return(dist);
```

```
}

// Returns whichever color is picked up the most by the Color Sensor
// This is
String get_agreed_color(){
  String color1;
  String color2;
  String color3;
  bool same = false;

  while(same == false){
    color1 = get_colorsensor_color();
    delay(50);
    color2 = get_colorsensor_color();
    delay(50);
    color3 = get_colorsensor_color();
    delay(50);
    if((color1 == color2 && color2 == color3) && color1 != "None"){
      same = true;
    }
  }

  return color1;
}

// Returns the Color Detected by the Color Sensor as a String (Takes 0.3sec)
String get_colorsensor_color(){
  digitalWrite(S2, LOW); //Setting red filtered photodiodes to be read
  digitalWrite(S3, LOW);
  redRaw = pulseIn(sensorOut, LOW); //Reading the output frequency
  Serial.print("R= "); //Printing the value on the serial monitor
  Serial.print(redRaw);

  Serial.print(" ");
  delay(100);

  digitalWrite(S2, HIGH); //Setting Green filtered photodiodes to be read
  digitalWrite(S3, HIGH);
  greenRaw = pulseIn(sensorOut, LOW); //Reading the output frequency
  Serial.print("G= "); //Printing the value on the serial monitor
  Serial.print(greenRaw);

  Serial.print(" ");
  delay(100);

  digitalWrite(S2, LOW); // Setting Blue filtered photodiodes to be read
```

```
  digitalWrite(S3, HIGH);
  blueRaw = pulseIn(sensorOut, LOW); // Reading the output frequency
  Serial.print("B= "); // Printing the value on the serial monitor
  Serial.println(blueRaw);

  Serial.println(" ");
  delay(100);

  // Using redRaw, greenRaw, and blueRaw we guess the Color of the cube
  // Notes:
  // - colorRaw < 100 works as the proximity threshold, could be changed to allow slightly further
distance from the sensor
  // - Always leave whichever color gets detected the most last, for in this case, that is redRaw
  if(redRaw < 60 && greenRaw > blueRaw && redRaw < blueRaw){
    Serial.println("It's Orange");
    return "Orange";
  }
  else if(greenRaw < 100 && redRaw < greenRaw && redRaw < blueRaw){
    Serial.println("It's Yellow");
    return "Yellow";
  }
  else if(greenRaw < 125 && greenRaw < redRaw && blueRaw < greenRaw && blueRaw >
80){
    Serial.println("It's Green");
    return "Green";
  }
  else if(blueRaw < 100 && blueRaw < redRaw && blueRaw < (greenRaw - 10)){
    Serial.println("It's Blue");
    return "Blue";
  }
  else if(redRaw < 100 && redRaw > 50){
    Serial.println("It's Red");
    return "Red";
  }
  else{
    Serial.println("It's Nothing!");
    return "None";
  }
}
```

References

1. Circuit Diagram. (2019). *Circuit Diagram - A Circuit Diagram Maker*. Circuit-Diagram.org. https://www.circuit-diagram.org/

2. DroneBot Workshop. (2020, January 19). *Arduino Color Sensors - TCS230 & ISL29125*. Www.youtube.com. https://www.youtube.com/watch?v=MwdANEcTiPY

3. DwyerOmega. (2023, September 5). *The Load Cell: What it is, What it Does, How it Works!* YouTube. https://www.youtube.com/watch?v=kRDQ4oYWUjM

4. GreatScott! (2017). Electronic Basics #33: Strain Gauge/Load Cell and how to use them to measure weight [YouTube Video]. In *YouTube*. https://www.youtube.com/watch?v=lWFiKMSB_4M

5. Rice Lake Weighing Systems. (2020, November 16). *Five Types of Load Cells*. YouTube. https://www.youtube.com/watch?v=dISwdhWCNgw